

**SIMATS ENGINEERING**  
**CO2 - ASSESSMENT TOOL 2**

**Scenario-Based Questions**

**COURSE NAME: Deep Learning**

**COURSE CODE: MLA0408**

**COURSE OUTCOME COVERED**

**CO 2:** Students will be able to apply supervised learning algorithms such as linear regression, classification models, decision trees, neural networks, support vector machines, and ensemble methods to solve real-world prediction and classification problems. They will gain the ability to select appropriate models, train them using datasets, and evaluate their performance. Students will also understand the strengths and limitations of different supervised learning approaches. (BTL-4)

**CO Weightage: 15%**

**Assessment Tool Weightage: 40%**

**Duration: 90 minutes**

**Marks: 25**

**Code of Conduct:**

I B.Jahnavi / 192325043 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

**Signature of the student: B.Jahnavi**

## Scenario-Based Questions

CO-AT2

QUESTIONS: 05

Total Marks:25

### 1.Unsupervised Training of Neural Networks, Auto Encoders

A hospital wants to identify abnormal ECG patterns from thousands of unlabeled records. Which deep learning technique from the syllabus would you recommend? Justify your choice and explain the workflow.

#### Recommendation: Autoencoders

An **Autoencoder** is an unsupervised neural network that learns to compress the input data into a lower-dimensional representation (encoding) and then reconstruct it back to its original form. Since the ECG records are **unlabeled**, autoencoders are ideal because they do not require class labels during training.

#### Justification

- The hospital has **thousands of unlabeled ECG signals**, making supervised learning unsuitable.
- Autoencoders learn the **normal characteristics** of ECG signals by minimizing reconstruction error.
- When an **abnormal ECG** is given as input, the network cannot reconstruct it accurately because it differs from the learned normal patterns.
- These abnormal records produce a **high reconstruction error**, which can be used to detect anomalies.

#### Workflow

##### 1. Data Collection

- Gather thousands of unlabeled ECG recordings.
- Preprocess the data (remove noise, normalize, segment ECG signals).

##### 2. Training the Autoencoder

- Input ECG signals into the autoencoder.
- **Encoder:** Compresses the ECG into a compact feature representation.
- **Decoder:** Reconstructs the ECG from the encoded features.
- The network learns by minimizing the reconstruction loss (difference between original and reconstructed ECG).

##### 3. Learning Normal Patterns

- Since most ECG records are assumed to be normal, the autoencoder learns the common structure of normal heart signals.

##### 4. Anomaly Detection

- Feed a new ECG record into the trained autoencoder.
- Calculate the reconstruction error.
- If the error is **below a threshold**, classify it as **normal**.
- If the error is **above the threshold**, classify it as **abnormal** and flag it for medical review.

## Advantages

- Works without labeled training data.
- Detects unusual or rare ECG patterns effectively.
- Reduces manual effort for cardiologists by automatically screening large datasets.
- Learns meaningful feature representations of ECG signals.

## Conclusion

**Autoencoders** are the recommended deep learning technique because they perform **unsupervised learning** and are highly effective for **anomaly detection** in large collections of unlabeled ECG data. By learning normal ECG patterns and identifying records with high reconstruction error, they can accurately flag potentially abnormal heart conditions for further clinical analysis.

## 2.Auto Encoders, Representation Learning

**A retail company wants to reduce the dimensionality of customer purchase data before performing customer segmentation. Explain how an autoencoder can be used to solve this problem.**

### Autoencoders for Representation Learning

A retail company wants to reduce the dimensionality of customer purchase data before performing customer segmentation. An Autoencoder is an effective deep learning technique for representation learning because it learns a compact representation of high-dimensional data without requiring labeled data.

#### How an Autoencoder Solves the Problem

##### Input Layer

Customer purchase data (e.g., products bought, purchase frequency, spending, visit history) is fed into the autoencoder.

##### Encoder

The encoder compresses the high-dimensional customer data into a smaller set of features called the latent representation or bottleneck layer.

This representation captures the most important purchasing patterns while removing redundant information.

##### Latent Representation

The compressed features serve as a low-dimensional representation of each customer.

These features preserve meaningful information needed for analysis.

##### Decoder

The decoder reconstructs the original purchase data from the compressed representation.

The network is trained to minimize the difference between the original and reconstructed data (reconstruction error).

##### Customer Segmentation

After training, only the encoder is used.

The compressed feature vectors are given to clustering algorithms such as K-Means or Hierarchical Clustering to group customers with similar purchasing behavior.

#### Workflow:

Customer Purchase Data → Encoder → Compressed Features (Latent Representation) → Decoder → Reconstructed Data

### **After training:**

Customer Purchase Data → Encoder → Compressed Features → Customer Segmentation (Clustering)

### **Advantages**

Reduces dimensionality while preserving important information.  
Removes redundant and noisy features.  
Improves the efficiency and accuracy of clustering algorithms.  
Handles large, complex customer datasets effectively.  
Learns features automatically without manual feature engineering.

### **Conclusion**

An Autoencoder reduces the dimensionality of customer purchase data by learning a compact latent representation. These compressed features are then used for customer segmentation, resulting in faster, more accurate clustering and better identification of customer groups for targeted marketing and business decisions.

## **3.Width and Depth of Neural Networks, Activation Functions**

**A facial recognition system performs poorly when trained using a shallow neural network. Recommend architectural changes involving network depth and activation functions.**

### **Width and Depth of Neural Networks, Activation Functions**

A facial recognition system performs poorly when trained using a shallow neural network. To improve its performance, the following architectural changes are recommended.

#### **1. Increase Network Depth**

Replace the shallow network with a deep neural network having multiple hidden layers.  
Deeper networks learn hierarchical features:  
First layers: Detect edges and textures.  
Middle layers: Detect facial features such as eyes, nose, and mouth.  
Final layers: Learn complete face representations for accurate recognition.  
This improves the model's ability to capture complex facial patterns.

#### **2. Increase Network Width (if required)**

Increase the number of neurons in hidden layers to improve the model's learning capacity.  
Avoid making the network excessively wide, as it may increase computation and lead to overfitting.

#### **3. Use Appropriate Activation Functions**

##### **ReLU (Rectified Linear Unit):**

Most commonly used for hidden layers.  
Formula:  $\text{ReLU}(x) = \max(0, x)$

##### **Advantages:**

Faster training.  
Reduces the vanishing gradient problem.  
Improves convergence.

##### **Leaky ReLU (optional):**

Allows a small gradient for negative inputs.  
Helps avoid the "dying ReLU" problem.

##### **Softmax:**

Use in the output layer for multi-class facial recognition.  
Produces probabilities for each person's identity.

#### **4. Additional Improvements**

Apply Batch Normalization to stabilize and speed up training.

Use Dropout to reduce overfitting.

Train with a larger and more diverse dataset for better generalization.

#### **Summary of Recommended Architecture**

**Input Image** → Deep Hidden Layers (ReLU/Leaky ReLU) → Batch Normalization + Dropout (optional) → Output Layer (Softmax)

#### **Advantages**

Learns complex facial features more effectively.

Improves recognition accuracy.

Reduces vanishing gradient issues.

Faster and more stable training.

Better generalization to new faces.

#### **Conclusion**

A deep neural network with multiple hidden layers and ReLU (or Leaky ReLU) activation functions is recommended for facial recognition. Use Softmax in the output layer for classification. This architecture extracts richer facial features, improves learning efficiency, and significantly enhances recognition accuracy compared to a shallow neural network.

#### **4.Restricted Boltzmann Machines (RBMs), Unsupervised Training of Neural Networks**

**A streaming service wants to recommend movies to users based on their viewing history without using labeled data. Explain how RBMs can be applied to build the recommendation engine.**

#### **Restricted Boltzmann Machines (RBMs) – Movie Recommendation System**

A streaming service wants to recommend movies to users based on their viewing history without using labeled data. A Restricted Boltzmann Machine (RBM) is a suitable unsupervised learning technique for this task because it learns hidden patterns in user preferences.

#### **How RBMs Build the Recommendation Engine**

##### **Prepare the Data**

Create a user–movie matrix.

Rows represent users, columns represent movies.

Each entry indicates whether a user has watched, liked, or rated a movie.

##### **RBM Architecture**

Visible Layer: Represents users' movie preferences (watched/rated movies).

Hidden Layer: Learns latent features such as movie genres, actors, or viewing interests.

There are connections only between the visible and hidden layers (no connections within the same layer).

##### **Unsupervised Training**

Train the RBM using the user–movie matrix.

The RBM learns hidden patterns by adjusting weights to minimize reconstruction error (typically using Contrastive Divergence).

No labeled data is required.

#### **Learning User Preferences**

Hidden neurons capture similarities between users and movies.

Users with similar viewing patterns develop similar hidden representations.

### **Generating Recommendations**

For a user, provide their viewing history as input.

The RBM reconstructs the missing movie preferences.

Movies with the highest predicted preference scores that the user has not watched are recommended.

### **Workflow**

User–Movie Matrix → RBM (Visible Layer → Hidden Layer) → Learn Hidden Features → Reconstruct Preferences → Recommend Unwatched Movies

### **Advantages**

Works effectively with unlabeled data.

Learns hidden relationships between users and movies.

Handles sparse user-rating matrices.

Improves personalized recommendations.

Discovers latent features automatically without manual feature engineering.

### **Conclusion**

A Restricted Boltzmann Machine (RBM) is an effective unsupervised learning model for movie recommendation systems. By learning hidden patterns from users' viewing histories and reconstructing missing preferences, it can recommend movies that match each user's interests without requiring labeled training data.

## **5. Deep Learning Applications, Representation Learning**

**A manufacturing company wants to detect defective products from sensor readings collected every second. Design a deep learning solution using concepts from representation learning and neural networks.**

### **Deep Learning Applications – Representation Learning**

A manufacturing company wants to detect defective products from sensor readings collected every second. A deep learning solution using representation learning and neural networks can automatically learn important features from sensor data and identify defective products accurately.

#### **Proposed Solution**

##### **1. Data Collection**

Collect sensor readings such as temperature, pressure, vibration, and speed every second.

Preprocess the data by removing noise, handling missing values, and normalizing the readings.

##### **2. Representation Learning**

Use an Autoencoder to learn a compact (latent) representation of the sensor data.

The encoder extracts the most important features, while the decoder reconstructs the original sensor readings.

This reduces dimensionality and removes redundant information.

### **3. Neural Network for Defect Detection**

Feed the learned feature representation into a Deep Neural Network (DNN).

The DNN learns patterns that distinguish normal and defective products.

If labeled defect data is unavailable, use the reconstruction error of the autoencoder:

Low reconstruction error → Normal product

High reconstruction error → Defective product

### **4. Prediction**

For each new product, sensor readings are processed through the trained model.

The system classifies the product as Normal or Defective.

Defective products are flagged for inspection or removed from the production line.

### **Workflow**

**Sensor Readings** → Data Preprocessing → Autoencoder (Representation Learning) →

**Compressed Features** → Deep Neural Network / Reconstruction Error → Normal or Defective Product

### **Advantages**

Automatically learns useful features from raw sensor data.

Reduces dimensionality and noise.

Detects defects with high accuracy.

Works well with large-scale, continuous sensor data.

Enables real-time quality monitoring and reduces manual inspection.

### **Conclusion**

A deep learning solution combining representation learning (Autoencoder) and a Deep Neural Network provides an effective method for defect detection. The autoencoder learns meaningful representations of sensor data, while the neural network (or reconstruction error) identifies defective products, improving manufacturing quality and efficiency.

**RUBRICS FOR CALCULATION:**

| <b>Criteria</b>           | <b>Weightage</b> | <b>Level 4 – Exemplary</b>                                                  | <b>Level 3 – Proficient</b>                       | <b>Level 2 – Developing</b>                                 | <b>Level 1 – Beginning</b>              |
|---------------------------|------------------|-----------------------------------------------------------------------------|---------------------------------------------------|-------------------------------------------------------------|-----------------------------------------|
| Concept Identification    | 25%              | Identifies all key concepts from literature accurately and comprehensively  | Identifies most key concepts with minor omissions | Identifies limited concepts; some are irrelevant or missing | Fails to identify essential concepts    |
| Linkages                  | 25%              | Establishes clear, meaningful, and accurate relationships between concepts  | Shows correct relationships with minor gaps       | Relationships are weak, unclear, or partially incorrect     | No meaningful relationships established |
| Organization              | 20%              | Concepts are well-structured hierarchically with logical flow and grouping  | Mostly organized with minor structural issues     | Some organization but lacks clear hierarchy                 | No logical organization or structure    |
| Integration of Literature | 20%              | Integrates multiple studies effectively to show connections and comparisons | Shows some integration of literature              | Limited integration; mostly isolated concepts               | No integration of literature evidence   |
| Clarity                   | 10%              | Concept map is clear, neat, visually structured, and easy to interpret      | Generally clear with minor readability issues     | Clarity is reduced; difficult to interpret in parts         | Poorly presented and hard to understand |